

From greedy fill to monotone matching

How first pass chooses item identities, then and now

Explainer for exfret's property randomizer (`propertyrandomizer`)

26 September 2026

Summary

Short answer: a new algorithm for the same problem

The *problem* first pass solves hasn't changed. It still splits each item into a *position* (slot) and an *identity* (trav) and picks a one-to-one assignment of identities to positions of the same type, within a cost window. Item reflection then turns that assignment into the game, and promotion reasons over the same split graph.

The *search* that picks the assignment is new. None of the greedy fill's machinery survived: its reachability tests, reservations, target mechanic, swap chains and "desperation" mode are all gone.

- **Old (greedy forward fill):** built the assignment one pair at a time, in the order of an imagined playthrough, starting from nothing. When it got stuck, the attempt failed.
- **New (monotone matching):** starts from the vanilla assignment, which is valid by definition. It then replaces the *whole* assignment three times, each time with a random one that keeps every protected context. When a proposal fails the check and can't be repaired, it keeps the assignment it already had.

It's new in a second sense too. First pass now reasons the way promotion does, with pebbles, ranks, backings that go strictly earlier, and a fallback that's always available. So unified randomization rests on one theory instead of two.

The four changes that matter:

1. **From building to moving.** The greedy fill grew a partial assignment and could reach a dead end. Monotone matching always holds a complete, valid assignment and moves from one valid assignment to another (Proposition 3.1).
2. **From "reachable" to "reachable in the right contexts".** The greedy asked whether a slot or identity had *some* context. The new first pass asks, for each protected pebble (node, room, abilities), whether it's still there, and checks that exactly with a fresh sort (the *gate*).
3. **From reservations to proofs.** The greedy steered by holding back identities that didn't help the next mechanic on the science-pack path. The new first pass proves every protected pebble in a random sort and pins an identity only in the contexts where the proof uses it. Every other identity may go anywhere its type and cost allow.
4. **The rank rule.** A needed identity may only move to a position that, in each context it's needed in, is reached *before* the identity was reached in the current sort. Needed identities therefore only move earlier, which is where the name *monotone* comes from.

	Greedy forward fill (until 26 Sep)	Monotone matching (now)
Starts from	the empty assignment	the vanilla assignment
Unit of choice	one slot/identity pair at a time	the whole matching, per round
Validity test	slot or identity has <i>some</i> context	every hard pebble kept, checked by a fresh sort
Steering	reservations toward the next targeted mechanic	needs from proofs, plus the rank rule
What's protected	targeted mechanics, by node, once reached anywhere	all protected mechanic pebbles, planet-locked recipes, and every recipe reachable
When stuck	retry below 50%, else turn off the checks	keep the previous matching (always valid)
Measured losses	lost isolatable contexts on every measured seed	0 protected contexts lost by first pass
Identities moved	not measured	237–247 of 255 per round

Table 1: At a glance. The greedy's measurements are from the 25 September investigation (Section 2.2); the new ones are from 26 September runs (Section 5).

Contents

Summary	1
1 The problem both versions solve	3
1.1 Positions and identities	3
1.2 The assignment is a perfect matching	3
1.3 What a valid assignment must keep	5
2 The old approach: a greedy forward fill	5
2.1 How it worked	6
2.2 Why it lost contexts	6
3 The new approach: monotone matching	8
3.1 The idea: move between valid matchings	8
3.2 One round	9
3.3 After the rounds	12
4 Side by side: is it a completely new approach?	13
4.1 What was kept, changed, removed and added	13
4.2 The answer	13
5 Measurements	14
6 Limits and open points	16
A Where things are in the code	16
B Glossary	17

1 The problem both versions solve

First pass runs inside unified randomization (`randomizations/graph/unified/execute-new.lua`), before the other handlers. It fixes item randomization’s permutation, which says which item *identity* each item *position* gets. The handlers that run after it (promotion, recipe ingredients, and the rest) then reason over the graph first pass built. This section covers what stayed the same from the greedy fill to monotone matching.

1.1 Positions and identities

For every node that takes part, first pass splits the logic-graph node into two nodes (`first-pass-new.lua`, “SPLIT GRAPH NODES”):

Slot (the position).

It keeps the node’s name and the edges that say *where the item comes from* and *which recipes use it*. These are crafting, loot, burnt results, items that spoil into it, resource and asteroid mining, and recipe ingredient uses (the `always_slot_pre` and `always_slot_dep` tables in `compat/vanilla.lua`). Any edge a handler randomizes (a base or head) also stays on the slot.

Trav (the identity), named X-trav.

It gets every other fixed edge, which says *what the item is*: it places a building, it’s a fuel, it’s launched and delivered. It also gets sources that follow the item’s name. In the model, mining a building or a tile gives the item under its own name (`dutils.mining_keeps_item_names`), so that edge belongs to the identity. Entity randomization’s identity heads and bases move here too.

The edge `slot → trav` is then cut into a base (on the slot) and a head (on the trav). Choosing the assignment means choosing which base connects to which head. Items get one extra edge, `trav → slot`. After reflection, the item made at a position *is* the identity’s item, so the position’s consumers also get the identity’s own sources, such as delivery to other rooms or spoiling (`connect` in `monotone-matching.lua`).

Which nodes take part. `valid_node_for_first_pass` leaves out spoofed and blacklisted nodes, resource mining (`entity-mine`), science packs (lab inputs), recipes, technologies and `entity-operate`. A node takes part if a handler randomizes one of its prerequisites (it has a head), or if it’s an item and item randomization is on. In all three measured seeds, every one of the 255 slots was an item (Section 5). Entity randomization is adding *entity positions* to first pass (the `first_pass_entity` flag). That work was in progress and uncommitted when this was written.

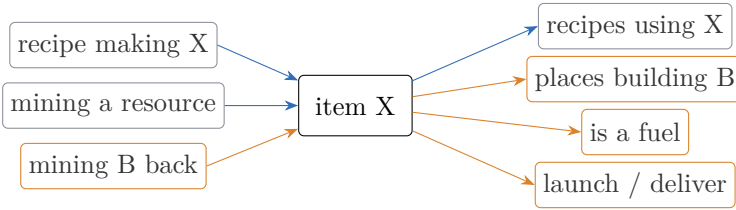
Reflection is the model. Item reflection (`item.reflect` in `handlers-new/item.lua`) makes the change real. The item made at position *s* becomes identity *t*’s item, in the recipes that make it and in the recipes that use it. There is one exception: reflection doesn’t swap two useless items (`dutils.reflected_item_position`). The new first pass gates exactly the matching reflection will realize (Section 3.2.5).

1.2 The assignment is a perfect matching

Let $\mathcal{S}$ be the slots and $\mathcal{T}$ the travs, one per slot. An *assignment* is a bijection $M : \mathcal{S} \rightarrow \mathcal{T}$ that pairs only nodes of the same type, and only pairs that `pair_ok` allows:

- **Cost window:** for items, $\ln c(t) - \ln c(s) \leq 5$ and $\ln c(s) - \ln c(t) \leq 8$, where *c* is the material cost (`first_pass_max_cost_log_difference_expensive` and `_cheap` in `helper-tables/constants.lua`). An identity can go into a position that’s much more expensive than it more easily than the reverse.

(a) Logic graph



(b) Split graph built by first pass

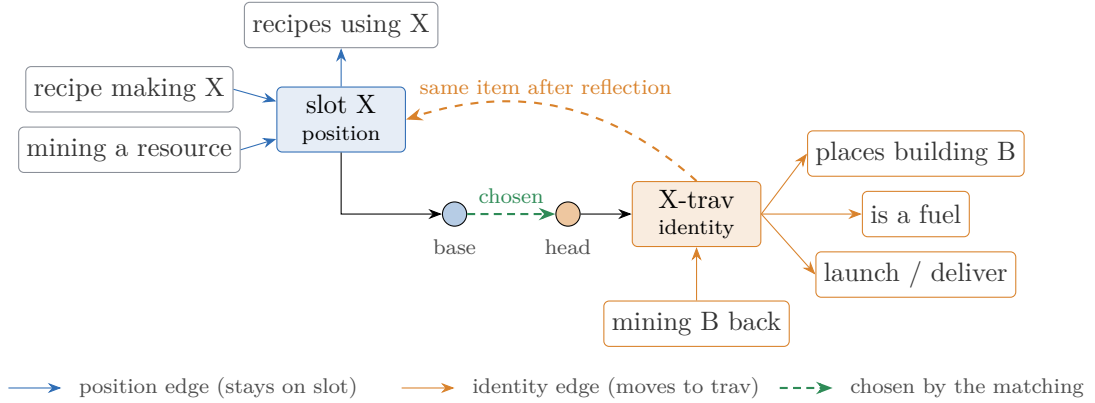


Figure 1: How first pass splits an item. The matching reconnects each slot's base to some trav's head. With the vanilla (identity) matching, (b) behaves like (a). Coal is a special case: item reflection makes whatever replaces coal a chemical fuel, so that fuel edge stays on coal's *slot*, and coal's identity gets a fuel edge of its own (`dutils.replacement_gets_fuel`).

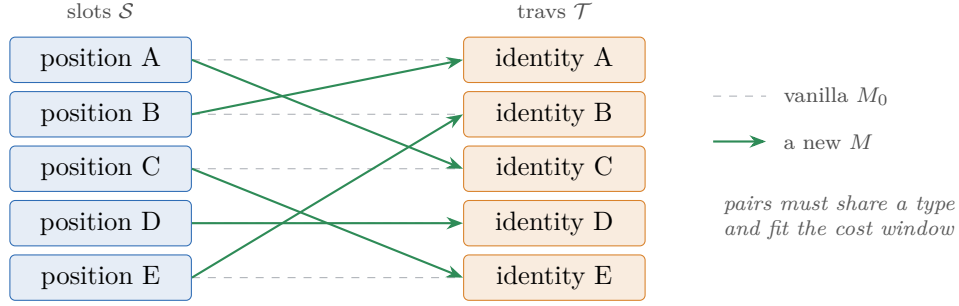


Figure 2: The assignment as a bipartite perfect matching. An identity may also stay where it is (D here).

- **Coal’s position** only takes identities with no burnt result (`pair_ok`).

The vanilla game is the *identity matching* $M_0(s) = s$ ’s own trav. Both algorithms output an M . What differs is how they find it and what they check.

1.3 What a valid assignment must keep

Pebbles and contexts. A *complex sort* (`lib/graph/context-sort.lua`, with `complex_contexts` and `home_contexts`) reaches nodes in *contexts*. A context is a room together with an ability string for isolatability and automatability, written like `planet:vulcanus | 11`. A *pebble* $\langle v, c \rangle$ is a node v reached in context c , and its *rank* $\text{rk}\langle v, c \rangle$ is its position in the sort. “Earlier” always means lower rank.

Hard pebbles. What must survive first pass is decided in one place, `randomizations/graph/unified/skeleton/protection.lua`:

1. **Mechanic pebbles.** Their room and automatability are always kept. Isolatability is kept for nodes that declare `keep_isolatability=true`, or for every node with `constants.keep_isolatability`. An unprotected isolatable pebble only needs its non-isolatable counterpart (`is_hard_mechanic_pebble`, `kept_part`).
2. **Planet-locked recipes** keep every context they have on their planet (`planet_locked_recipe_contexts`).
3. **Every recipe** that was reachable stays reachable, in at least one context.

Home contexts are never kept for their own sake; they only back the discovery rule. In the measured seeds, the sort had about 275,000 pebbles. Of those, 3,527–3,548 had to be kept exactly and 619–621 recipes had to stay reachable.

This definition of “valid” is itself newer than the greedy fill, which never had it: `protection.lua` landed in the same commit as monotone matching (014b8be). Section 2 shows the greedy’s own, weaker notion.

2 The old approach: a greedy forward fill

This was first pass from August (bd888e5, “Work on mechanics-aware reservation system”, 7 August) until 26 September. That day 014b8be put monotone matching into the experimental copy, and b717eda deleted the greedy paths 17 minutes later. Its last version is `git show b717eda^:randomizations/graph/unified/first-pass-new.lua` (1,862 lines). In item-randomizer terms it was a *forward fill*: simulate a playthrough, and at each step put something into a location that’s already reachable.

2.1 How it worked

Setup.

1. Take a random sort of the vanilla graph, with plain room contexts (no abilities), and order the slots by their first rank.
2. *Targets*: take the witness (`top.path`) of the starting planet’s science packs. The mechanic nodes on it (plus nodes flagged **important**), in sort order, form a list, and the first one not yet reached is the *current mechanic*.
3. For each identity, record the mechanics it reaches first: a breadth-first search to depth 20 that doesn’t pass through mechanics.
4. Build the split graph (Figure 1) with every slot/trav connection cut, and sort it.

Its notion of “reachable”. A slot counted as reachable if it’s an OR node with a head prerequisite or any prerequisite with *some* context, or an AND node whose prerequisites share *some* context (`slot_absolute_reachable`). An identity counted as reachable if it’s an OR node (always), or an AND node whose fixed prerequisites share some context (`trav_absolute_reachable`). Both are yes/no questions about whether there’s any context at all.

Main loop (once per slot, Figure 3).

1. Take the earliest unassigned, reachable slot.
2. Walk the free, reachable identities in one fixed shuffled order and take the first *compatible* one (`is_compatible`). It must be the same type, have the same first-reached mechanics (for non-items), and fit the cost window (for items). For ore slots, a useless identity was rejected 90% of the time.
3. If the identity helps the current mechanic, *connect* it: add `base → head` and update the sort incrementally. The current mechanic moves on as soon as it has any context. Otherwise, *reserve* it: record the pair but don’t connect it, so it adds nothing to the world yet.
4. If no pair fits, run the *reservation loop*:
 - (a) connect a reserved pair whose identity helps the current mechanic;
 - (b) else swap a free helping identity into a reserved slot;
 - (c) else build a chain of swaps through reserved slots;
 - (d) else connect the oldest reservation anyway.

Then try again, up to 10,000 times per slot.

5. If it’s still stuck: below 50% of slots assigned, the attempt fails and unified retries. Above 50%, it turns the reachability checks off (“desperation”) and continues. If it gets stuck again, it fails.

End. Connect the remaining reservations, sort again, and return. The committed version had no check of what the result kept. The experimental copy added one (`GATE_MECHANIC_CONTEXTS`) together with complex contexts, and that check is what exposed the losses below.

2.2 Why it lost contexts

1. **Yes/no reachability.** A slot that has *some* context passes, even if it never has the context the identity is needed in. There are two ways this goes wrong (Figure 4). In a *static mismatch*, the slot is never reached in that context at all. In a *context-local cycle*, the slot is reached in that context only through the identity it’s about to hold.

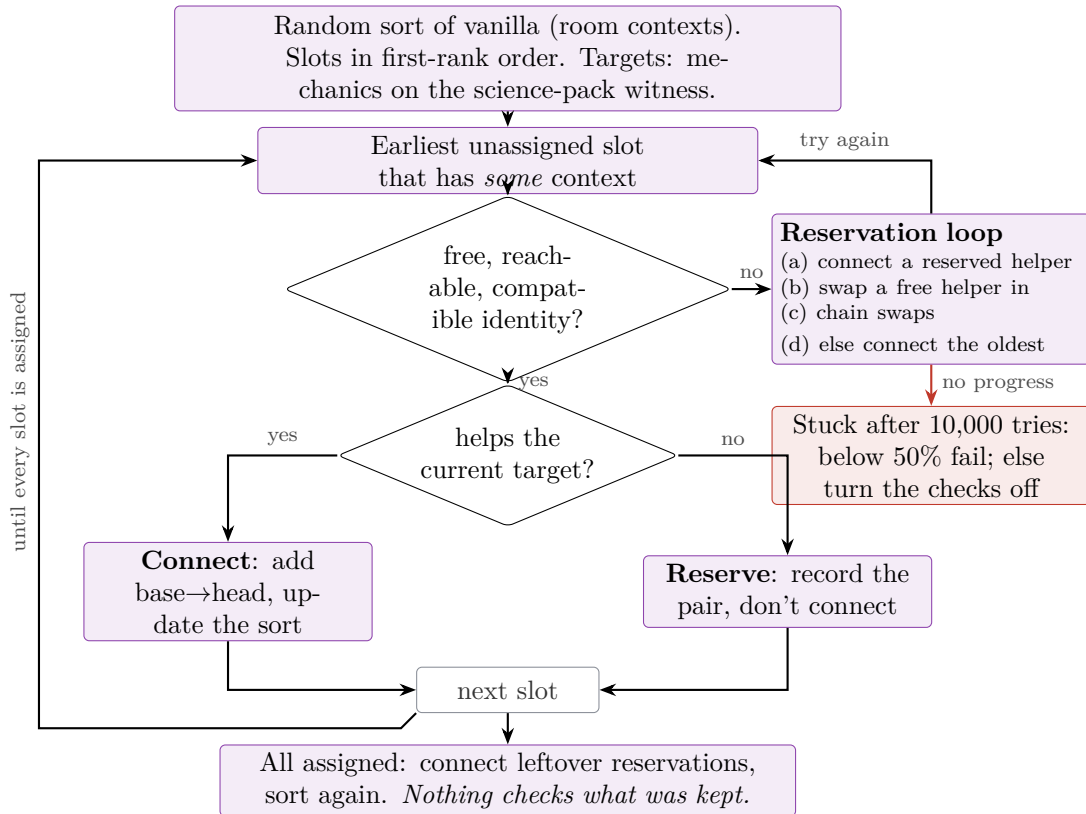


Figure 3: The greedy forward fill’s control flow (b717eda[^]). Reservations steered progress toward the next targeted mechanic, and the reservation loop was the only way out of a dead end.

2. **Targets by node, only on the science path.** A mechanic counted as done once it had *any* context, so a later mechanic that needed it in another room was never protected. Mechanics off the starting planet’s science-pack witness weren’t targeted at all. The experimental copy’s comment on `TARGET_ALL_MECHANICS` says “untargeted ones weren’t protected at all”.
3. **No fallback.** A partial assignment that can’t be extended is a dead end. The reservation loop only reshuffles pairs that are already placed; it never goes back to a known good state. Desperation mode fills the rest without checks.
4. **Reproducibility.** The ore roll used `math.random` rather than the mod’s seeded `rng`, and rolled again every time the same pair was checked. The experimental copy’s fix comment records this.

Measured on 25 September (experimental copy with complex contexts and a gate; from that investigation’s notes, not re-measured here):

- 7–12 misplaced identities per seed explained *all* of the roughly 466 lost mechanic pebbles, and every lost pebble was an isolatable context. The roots were static mismatches or context-local cycles. The notes’ example is coal’s identity landing in the roboport’s position, which broke a Vulcanus context.
- Adding a filter so the greedy only used slots reachable in the needed isolatable rooms made it stall at 77–81% on all 10 retries.

So the greedy couldn’t be patched to keep isolatability. Either it lost contexts or it got stuck.

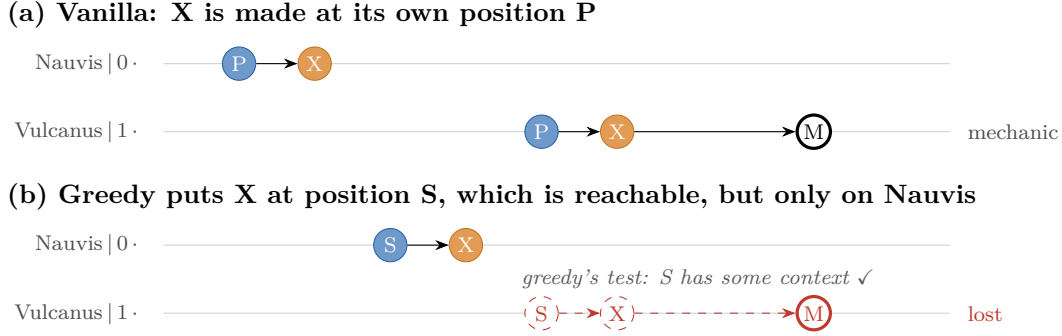
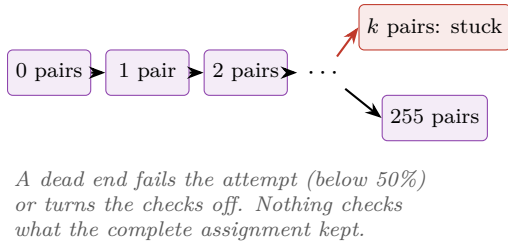


Figure 4: How yes/no reachability loses a context. The lanes are contexts (room|isolatability automatability) and rank grows to the right. In (b), S has no Vulcanus-isolatable pebble, so X doesn't either, and neither does the mechanic M that needed it. In a context-local cycle, S *would* reach Vulcanus, but only by using X there, so moving X to S breaks the loop.

(a) **Greedy: grow a partial assignment**



(b) **Monotone matching: valid to valid**

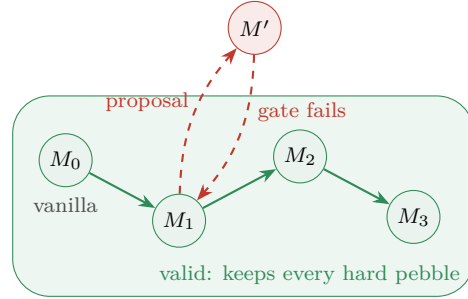


Figure 5: The two search spaces. In (b), every round ends at a valid matching: either the proposal passed the gate, or the round keeps the matching it started with. Failed proposals are rare: 4 of the 74 logged rounds (Section 5).

3 The new approach: monotone matching

Code: `randomizations/graph/unified/first-pass-new.lua` does the setup, the split and the final gate. `randomizations/graph/unified/skeleton/monotone-matching.lua` runs the rounds.

History: it landed in 014b8be (26 Sep, 00:31) and became the only first pass in b717eda (00:48). Mechanics-aware needs and the reflection model came in 85efe29, home contexts in 8f5e55d, and starting ores and coal's fuel in 86362b2.

3.1 The idea: move between valid matchings

The greedy fill searched through *partial* assignments, and most of them can't be completed in a way that keeps everything. Monotone matching searches through *complete, valid* assignments only. It starts from the one assignment known to be valid, vanilla, and makes big random moves. Each move is checked exactly, and a move that fails is simply not taken (Figure 5).

Proposition 3.1 (Every round ends valid). *Let $\mathcal{H}$ be the hard pebbles of the vanilla matching M_0 (Section 1.3), computed once before round 1. Assume the set of pebbles a complex sort reaches doesn't depend on the sort's random order. Then every round of `matching.run` returns a matching that keeps all of $\mathcal{H}$: each exact pebble in its context, and each recipe in at least one context.*

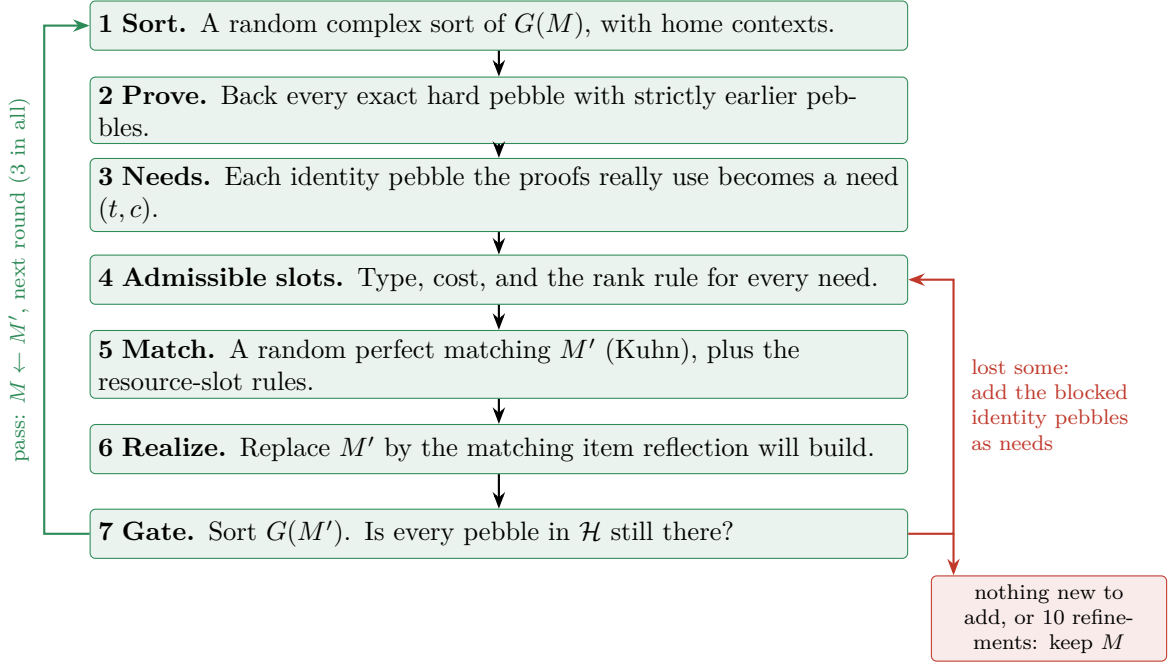


Figure 6: One round of `matching.run`. Steps 1–3 run once per round. Steps 4–7 repeat for each refinement. In 70 of 74 logged rounds, the first proposal passed. In the other 4, refinement found nothing to add, and the round kept M .

Proof. By induction on rounds. M_0 keeps $\mathcal{H}$, because $\mathcal{H}$ is read off M_0 ’s own sort. Round r returns one of two things. The first is a proposal whose fresh sort lost nothing in $\mathcal{H}$: that’s exactly the gate’s condition, and by the assumption, one sort speaks for all of them. The second, when refinement gives up, is the matching M_{r-1} the round started with. $\square$

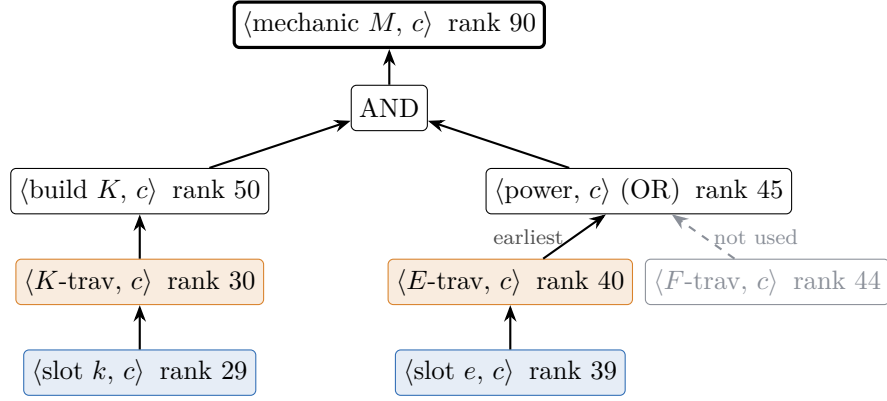
The assumption is what home contexts were introduced for. The old tech discovery rule depended on order, so two sorts of one graph could disagree. First pass also runs a second gate after the rounds, against the original graph (Section 3.3). So even if the assumption failed somewhere, a bad result would make the attempt retry rather than ship. Note what the proposition does *not* say: it doesn’t say a proposal is likely to pass. That’s the job of the proofs and the rank rule, and it’s measured rather than proved (Section 5).

3.2 One round

3.2.1 Steps 1–2: sort, then prove the hard pebbles

Each round takes a fresh random complex sort of the current graph $G(M)$. It then proves the *exact* hard pebbles in it: protected mechanic pebbles and planet-locked recipe pebbles. Recipes are left to the gate, since needs for them would be too strict. The prover (`make_prover`) builds a *backing* for each goal out of strictly earlier pebbles, like promotion’s `compute_support`:

- **AND node:** every prerequisite needs an earlier pebble that is itself established.
- **OR node:** the providers’ earlier pebbles are tried in rank order, earliest first. If one can’t be established, the next one is tried. This backtracking is what `top.path`’s single earliest-provider witness lacks.
- **Nodes that change contexts** (forgetters and emitters): try the incoming contexts that turn into this one, earliest first.
- **Technologies** can also be backed through the discovery rule (`top.discovery_candidates`): an earlier pebble of the tech itself in a home context, plus an earlier



*Needs: (K, c) and (E, c). F gets none from this proof, so it's free to move.
Slot pebbles are backings too, but only identity pebbles become needs.*

Figure 7: Needs come from a proof, not from every route. Power has two providers here. The prover takes the earliest one that it can itself prove (E), so only E is pinned in context c . The ranks are made up for the example.

pebble of a discoverer of the room.

Results are memoized. A pebble that's still being evaluated counts as unproven, so backings can't loop. The result is the *closure*: every pebble the proofs use. In the measured runs, every exact hard pebble was proven in every round, and the closure held about 26,000–28,500 pebbles.

3.2.2 Step 3: needs

A *need* is a pair (t, c) : identity t must still be available in context c . Needs come only from identity (trav) pebbles in the closure, and only from real uses (`needs_from_proof`, Figure 7):

- A trav pebble counts if some user in the proof leads to a real use of the identity. Users on the trav's own launch chain, or the trav itself, are followed further.
- *Delivered* contexts aren't needs. They're backed through the trav's own launch chain (trav $\rightarrow$ item-launch $\rightarrow$ item-deliver $\rightarrow$ trav), so they follow from the context the item was launched from.
- Sometimes a trav's only use in another room is feeding its own slot's consumers there, by delivery. Then the requirement moves to the *slot*: whatever identity it holds must be launchable (`requires_launchable`), and the identity itself isn't pinned.

Because the prover picks one provider per OR, an identity is pinned only where the proof actually uses it. If another provider comes first in the next round's sort, the old one is free again. With these rules, 90–100 of the 255 identities had needs in each measured round, with 215–228 needs between them. The other 155–165 identities were free to go anywhere their type and cost allowed.

3.2.3 Step 4: admissible slots, and the rank rule

Definition 3.2 (Admissible). Let rk be the ranks in this round's sort of $G(M)$. Slot s is *admissible* for trav t if s has the same type as t , `pair_ok(s, t)` holds, and

$$\forall (t, c) \in N : \quad \langle s, c \rangle \text{ exists and } \text{rk}\langle s, c \rangle < \text{rk}\langle t, c \rangle.$$

If a slot requires a launchable identity, t must also be launchable. The trav's current slot $M^{-1}(t)$ is always admissible.

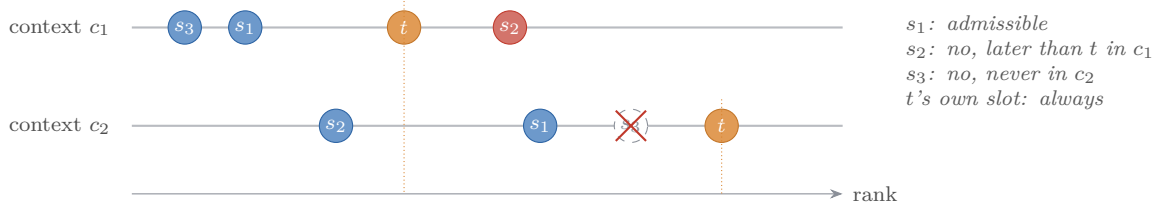


Figure 8: The rank rule for an identity t with needs in two contexts. A slot must beat t in *every* needed context.

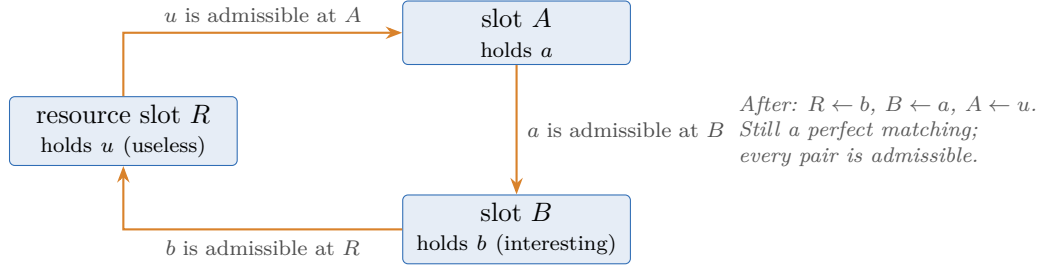


Figure 9: Cycle repair for a resource slot. The search is breadth-first from R over “the identity at this slot may move to that slot”.

In words, a needed identity may move only to a position that, in every context it’s needed in, was reached *before* the identity itself was reached through its current position. In the next graph, the identity’s supply in context c then comes from a pebble that was earlier than the identity. The proof’s backing through t only gets *earlier*, and that’s what *monotone* refers to. It’s the same “strictly earlier” condition that makes promotion’s promises well-founded.

In the measured rounds, identities with needs had on average 38–69 admissible slots besides their own, and free identities about 217. Between 2 and 9 identities per round had none besides their own, so they stayed put that round.

3.2.4 Step 5: a random perfect matching

The admissibility graph always has a perfect matching, because the current matching is one (each trav’s current slot is admissible). `random_matching` finds one with Kuhn’s augmenting-path algorithm. It visits the travs in random order and tries each trav’s admissible slots in random order, with its current slot *last*, which pushes identities to move. Augmenting paths find a perfect matching whenever one exists, so this step never fails.

Resource slots. Item reflection doesn’t swap two useless items, and ores count as useless, so an ore patch would usually keep mining what it did. The matching steers resource slots toward *new*, *interesting* identities. An identity is new and interesting for a resource slot if it’s not useless (`duils.is_useless_item`) and not the slot’s own identity.

1. A resource slot that already mines something new keeps its identity, so no round undoes it.
2. Every other resource slot, in random order, tries up to 5 random admissible new identities. It keeps one only if a perfect matching still exists with it.
3. After Kuhn’s algorithm finishes, a breadth-first search looks for a *cycle of moves* for each resource slot still without a new identity. Each identity on the cycle moves to a slot it’s admissible for, and the cycle ends with a new identity moving into the resource slot, without making another resource slot stop mining something new (Figure 9).

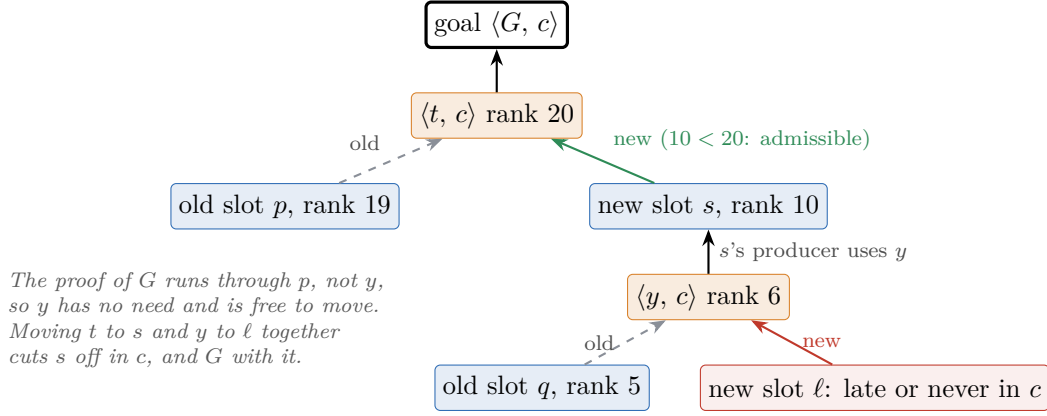


Figure 10: The gap the gate closes. Each move looks fine by itself, but together they break a slot’s own backing.

3.2.5 Steps 6–7: realize, gate, refine

Realize. Reflection places useless identities differently from the matching (`dutils.realized_item_assignment`), so the proposal is replaced by the matching reflection will actually build. That’s still a perfect matching, and realizing M_0 gives M_0 back. So the gate judges the game that will actually be built, not an idealized one. The same commit (85efe29) fixed a second place where the model and the game disagreed, in the split. Mining drops of player creations and of tiles now belong to the identity. Before that fix, the Fulgoran ruin’s stone brick drop was modeled as a position, which lost about 101 isolated-Fulgora contexts whenever that position held something else.

Gate. Connect M' , sort $G(M')$ from scratch, and list the hard pebbles that are missing (`lost_pebbles`). An exact pebble is lost if its context is missing, and a recipe is lost only if it has no pebble at all. If nothing is lost, the proposal is accepted.

Refine. Otherwise, for each lost pebble, the gate takes its witness in the round’s *starting* graph (`top.path`). The identity pebbles on that witness that the new sort no longer reaches become extra needs (`blocked_travs`). Then steps 4–7 run again with a new random stream. Needs only grow, and each trav’s current slot stays admissible whatever its needs, so a perfect matching still exists. If refinement can’t add a new need, or after 10 refinements, the round keeps M .

Why proposals can fail at all. The rank rule checks the moved identity’s needs, but not the new slot’s *own* backing. That backing may use other identities, and those may move too (Figure 10). The gate closes that gap exactly. Refinement closes it only when the missing piece shows up on the lost pebble’s old witness. In the failure drawn below, the witness runs through t ’s old slot, not through y , so refinement finds nothing new and the round keeps M . All four failed proposals in the logs ended exactly like that, with refinement adding no needs (Table 5).

3.3 After the rounds

`first_pass.execute` connects the final matching and sorts the split graph once more. It then runs its own gate against the *original* graph’s sort: every mechanic pebble keeps its protected part (`protection.kept_part`), and every planet-locked recipe keeps its contexts. If anything is lost, it returns `false` and unified randomization retries the attempt. Otherwise it returns the matching (`slot_to_trav`, `trav_to_slot`), the split graph, and its sort, which promotion reasons over. It also returns the planet-locked contexts and the “mechanics sets” that item reflection uses

to scale costs. That last piece is a leftover from the greedy: the depth-20 breadth-first search now only feeds cost scaling.

Work in progress when this was written (uncommitted, in another session):

- *Entity positions.* Entity randomization flags `entity-build-item` nodes as `first_pass_entity`, so first pass also decides which entity is built where an item used to place another one. Mine-back heads are *coupled* to their slot (`connect_extra`). Their cost rule uses the own item’s cost within `constants.entity_cost_tolerance`.
- *Debt goals.* With planetary changes superposed (`SUPERPOSED`, off by default), goals that only the older world reaches must stay reachable with its debt. They’re gated on a superposed sort and refined from its witnesses. This is step 2 of the plan in the context-shift report.

Neither changes the algorithm above. They add slots and goals to it.

4 Side by side: is it a completely new approach?

4.1 What was kept, changed, removed and added

Table 2 sorts every part of first pass into four groups: kept, changed, removed and new. It compares the last greedy version (`b717eda`) with the current working tree.

Kept	The slot/trav split and its edge rules (Figure 1). Which nodes take part. The first random sort of vanilla, and its science-tier variant for Pyanodon’s. The cost window, which moved from <code>is_compatible</code> ’s item branch into <code>cost_ok</code> . The depth-20 “mechanics reached first” search, which now only feeds item reflection’s cost scaling. The return values, so promotion and the handlers didn’t change.
Changed	Sorts now use room <i>and</i> ability contexts, plus home contexts. Ore slots used a 90% reject roll; now they keep new identities, get prefilled, and get cycle repair. The mining exception used to match <code>place_result</code> ; now it’s <code>dutils.mining_keeps_item_names</code> , shared with reflection. Coal used to be a commented-out special case; now its fuel stays on the slot, and <code>pair_ok</code> applies. Items get the trav → slot edge, first added in the experimental copy.
Removed	The reachability tests. Reservations and the target-mechanic list from the science-pack witness. The reservation loop and its swap chains. Desperation mode and <code>FAILURE_ACCEPTANCE</code> . Incremental sort updates while filling. <code>is_compatible</code> ’s same-mechanics and recipe checks. The recipe-first-pass branches (<code>SPECIAL_RECIPE_FIRST_PASS</code>). The debug walkers. <code>b717eda</code> removed 1,530 lines from <code>first-pass-new.lua</code> and added 121, and it deleted the 2,110-line experimental copy.
New	Hard pebbles from <code>protection.lua</code> , including planet-locked recipes. The backtracking prover and the needs taken from its proofs. The rank rule. The random perfect matching, with resource rules. Realizing the matching the way reflection will. The gate and refinement. Rounds that start from the last valid matching. First pass’s own final gate and retry.

Table 2: Code lineage from the greedy fill to monotone matching.

4.2 The answer

Yes, as a search algorithm. No, as a model of the problem. The *frame* is unchanged: what gets assigned, how the assignment is represented in the graph, and who consumes it. So downstream code (promotion, the handlers, item reflection) didn’t have to change when the algorithm was swapped. The *method* was replaced wholesale, not refactored, and no line of the greedy’s search loop survives.

It’s a different *kind* of algorithm, too:

- The greedy was a **simulation**. It played the game forward and placed things where the player could already reach. That’s the classic forward fill used by many item randomizers. Its correctness depended on the simulation’s notion of “reachable”, and that notion was coarser than what the randomizer has to keep.
- Monotone matching is a **certification**. It holds a world known to be completable (the current matching), a sort of it, and a fallback that’s always available (the same matching). It only accepts a change backed by strictly earlier pebbles, and then checked exactly.

That second description is also a description of *promotion* (`skeleton/promotion.lua`). Promotion’s fallback is a recipe’s vanilla ingredients. Monotone matching’s fallback is the matching it started the round with. Both reason in the same currency: pebbles, ranks, backings, and the protection rules. The context-shift report calls such a triple of world, sort and fallback a *reference*. In those terms, first pass used to be the one part of unified randomization that didn’t have one, and now it does.

	Promotion (recipes, generic handlers)	Monotone matching (first pass)
Reference world	first pass’s split graph, with its matching	the current matching M
Sort	one random complex sort	a fresh random complex sort per round
What must hold	promised pebbles are well-founded	hard pebbles of M_0 are kept
A change is proposed when	a candidate’s backing is strictly earlier	a needed identity’s new slot is strictly earlier
Fallback	the recipe’s vanilla ingredients	keep M
Exact check	UNIFIEDCHECK (built game)	the gate (per proposal), then first pass’s gate

Table 3: Monotone matching reuses promotion’s ideas. Both are “reference” constructions in the sense of the context-shift report.

5 Measurements

Setup. These runs used the working tree of 26 September 2026 (HEAD `fe064cb` plus uncommitted work), Space Age, and the user’s startup settings, on seeds 1–3. Measurement log lines were added to a *copy* of `monotone-matching.lua`. It ran in an overlay mod directory that symlinks every other file to the repo, so the live repo was never modified. The scripts and the extracted log lines are in `notes/first-pass-report/measure/`.

End to end. All three seeds passed both UNIFIEDCHECK and MECHCHECK. Seed 3 needed a second attempt. First pass passed its gate in both of seed 3’s attempts; the retry came from promotion, which couldn’t keep a promise for one recipe (fusion reactor equipment). Every one of the four first-pass runs had 255 slots, all items. There were 3,527–3,548 exact hard pebbles and 619–621 recipes, out of about 275,000 pebbles in the sort. The logs of the three multi-seed checks run earlier that day (`dev/check-seeds.py`, the same seeds, with 62 more rounds) agree. First pass’s own final gate never failed there either. The attempts that were retried failed later, on furnace recipe collisions caught by UNIFIEDCHECK or on a handler that couldn’t place something. In the latest of those runs, rounds took 2.6–11.4s, with a median of 5.0s, running three seeds in parallel.

seed	att.	rnd	needs (step 3)				adm. slots			moved		moved earlier (shift)				ores	ref.
			travs	needs	launch	pinned	needy	free	vs M_0	vs prev	needy	free					
1	1	1	96	225	44	3	66	216	241	241	67/88 (-0.11)	60/153 (+0.07)	8/8	0			
1	1	2	95	215	43	4	51	218	240	223	59/69 (-0.06)	64/154 (+0.04)	8/8	0			
1	1	3	93	217	39	7	45	220	247	226	55/70 (-0.04)	71/156 (+0.03)	8/8	0			
2	1	1	95	220	45	3	69	218	241	241	72/89 (-0.10)	61/152 (+0.07)	8/8	0			
2	1	2	100	225	40	5	42	217	244	226	61/79 (-0.03)	72/147 (+0.01)	8/8	0			
2	1	3	96	228	42	9	38	218	242	224	46/72 (-0.01)	76/152 (+0.01)	8/8	0			
3	1	1	90	223	44	4	66	217	239	239	61/79 (-0.12)	63/160 (+0.05)	7/8	0			
3	1	2	95	220	41	5	46	217	237	236	59/82 (-0.04)	62/154 (+0.04)	8/8	0			
3	1	3	94	216	39	5	40	218	242	225	47/74 (-0.01)	66/151 (+0.01)	8/8	0			
3	2	1	95	223	43	2	69	216	237	237	73/84 (-0.11)	61/153 (+0.05)	8/8	0			
3	2	2	92	228	40	6	47	217	242	226	53/71 (-0.04)	66/155 (+0.04)	8/8	0			
3	2	3	92	228	38	6	40	218	240	220	35/65 (-0.02)	82/155 (+0.00)	8/8	0			

Table 4: Every measured round. *travs/needs*: identities with needs, and how many needs they have. *launch*: slots that require a launchable identity. *pinned*: identities with no admissible slot but their own. *adm. slots*: the mean number of other admissible slots, for identities with needs and for free ones. *moved*: identities not at their vanilla position, and identities that changed slot this round. *moved earlier*: of the identities that changed slot, how many got an earlier first pebble. The shift is the mean change in first-pebble rank as a fraction of the sort’s length (negative is earlier). *ores*: resource slots that mine something new. *ref.*: refinements. Every exact hard pebble was proven in every round.

What the numbers say.

- **Freedom.** About 94% of identities (237–247 of 255) end a round away from their vanilla position, and 220–241 change slot in each round. The proofs pin 90–100 identities with 215–228 needs. Even those have 38–69 admissible slots on average; free identities have about 217. Only 2–9 per round had nowhere else to go.
- **Monotone in practice.** In round 1, 80% of the needed identities that moved got an earlier first pebble, with a mean shift of about -11% of the sort. Only 40% of the free ones did, with a mean shift of about $+6\%$. By round 3 both shifts are close to zero: the matching has settled, and later rounds reshuffle it rather than drift further.
- **The gate rarely has to act, and refinement never helped.** In all 12 measured rounds, the first proposal passed. In the multi-seed logs, 4 of 62 rounds had a proposal that lost hard pebbles (Table 5). All the lost pebbles were isolatable contexts of protected nodes. Each time, refinement found no identity pebble to add, and the round kept its matching. So the proofs and the rank rule are enough about 95% of the time, the gate catches the rest, and refinement as written hasn’t recovered a single round.
- **Ores.** All 8 resource slots mine something new by the end of round 2 in every run. Round 1 left one out once, and a later round’s cycle repair fixed it.
- **Launch rule.** 38–45 slots per round require a launchable identity instead of pinning theirs.

multi-seed run (26 Sep)	seed	round	lost	example lost pebble
19:01	3	3	127	create-platform @ planet: fulgora 10
19:11	2	2	48	create-platform @ planet: fulgora 10
19:19	2	2	1	item: utility-science-pack @ surface: space-platform 10
19:19	3	3	102	item: automation-science-pack @ planet: fulgora 10

Table 5: The four proposals that failed the gate in the day’s multi-seed logs. In every case refinement added 0 needs, and the round kept its matching. The 19:01 and 19:19 runs used slightly earlier versions of the working tree.

The greedy fill, for comparison. The only measurements of the greedy fill are from the 25 September investigation (Section 2.2). With complex contexts and a gate, it lost isolatable contexts on every measured seed: about 466 lost mechanic pebbles, explained by 7–12 misplaced identities per seed. With a filter to avoid those losses, it stalled at 77–81% on all 10 retries. At the time, monotone matching’s first version moved 176–224 of 255 identities, with no losses in its model. The current version moves more (237–247), because of the proof-based needs in 85efe29 and the ore rules in 86362b2. The greedy fill wasn’t re-run for this report. Its code no longer fits the current pipeline (home contexts, protection rules, entity randomization), so a fair re-measurement would need that version of the whole mod.

6 Limits and open points

1. **The proofs are a heuristic; the gate is the guarantee.** The rank rule checks a moved identity’s needs, but not its new slot’s own backing (Figure 10). A stronger rule would also require that backing to be proven and to use only identities that stay put. That’s safer per proposal, but it pins more identities. With about 5% of rounds lost to failed proposals (4 of 74 logged), it isn’t urgent.
2. **Refinement can come up empty.** It adds identity pebbles from the lost pebble’s *old* witness. If the break comes from a slot’s own backing, as in Figure 10, it adds nothing and the round keeps the previous matching. That’s safe, but the round’s randomness is wasted. All 4 failed proposals in the logs ended with refinement adding no needs (Table 5), which is what this case looks like, though their causes weren’t traced. One possible fix: also follow, in the old graph, the backing of each blocked identity’s *new* slot. That would reach the free identity that moved away (*y* in Figure 10) and pin it.
3. **One sort per round.** Needs and ranks come from one random sort. A different sort would pin different identities, which is why several rounds help. It also means “earlier” is only relative to that sort, not an absolute property of the game.
4. **Order independence is assumed.** Proposition 3.1 needs the reached pebble set to be the same for every sort order. Home contexts made the discovery rule order-independent. If another order-dependent rule ever appears in `context-sort.lua`, one sort might pass while another fails. First pass’s own gate and the later checks (UNIFIEDCHECK, MECHCHECK) would still catch that.
5. **Leftovers from the greedy.** `is_canonical_result` (with its “satellite” hotfix) and the mechanics-set search survive only for their callers: the recipe first pass handler and item reflection’s cost scaling. They aren’t part of the matching.
6. **Multipass.** The context-shift report found that first pass’s gate protects only what the raw world has. Its identity moves can therefore cut old-world routes that planetary randomization’s superposition would still need. The in-progress debt goals (Section 3.3) are the start of the fix: step 2 of that report’s plan, “first pass on the superposition, with two tiers”.

A Where things are in the code

Line numbers are from the working tree on 26 September 2026 (a snapshot taken while another session was editing both files).

Location	What it does
<code>randomizations/graph/unified/first-pass-new.lua</code>	
<code>cost_ok, pair_ok</code> (83–116)	the cost window, coal’s slot, entity positions’ own-item costs
<code>first_pass.execute</code> (119)	entry point, called from <code>execute-new.lua</code> (“FIRST PASS”)
<code>init sort</code> (129–187)	random complex sort of vanilla (plus Pyanodon’s science tiers)
<code>valid_node_for_first_pass</code> (193)	which nodes take part
<code>SPLIT GRAPH NODES</code> (242)	slot/trav split, base/head connectors, coal’s fuel edge
<code>MECHANICS</code> (341)	depth-20 mechanics-reached-first search (for item reflection’s costs)
<code>MATCHING</code> (428)	resource items, coupled heads, the call to <code>monotone_matching.run</code>
<code>final gate</code> (526 on)	kept parts of mechanic pebbles, planet-locked contexts, debt goals
<code>randomizations/graph/unified/skeleton/monotone-matching.lua</code>	
<code>connect</code> (36)	slot base $\rightarrow$ trav head, plus trav $\rightarrow$ slot for items
<code>hard_pebbles, lost_pebbles</code> (53, 97)	what the gate checks
<code>make_prover</code> (130)	backtracking prover with strictly earlier backings
<code>launch_chains, needs_from_proof</code> (319, 348)	needs, delivery, launchable slots
<code>random_matching</code> (426)	admissibility, Kuhn’s algorithm, resource rules, cycle repair
<code>blocked_travs</code> (613)	refinement: blocked identity pebbles on old witnesses
<code>realize</code> (631)	the matching reflection realizes
<code>debt goals</code> (646–719)	superposed planetary goals (in progress)
<code>matching.run</code> (732)	rounds, the gate and refinement loop
Elsewhere	
<code>skeleton/protection.lua</code>	which pebbles are hard
<code>lib/data-utils.lua</code>	<code>mining_keeps_item_names</code> , <code>reflected_item_position</code> , <code>realized_item_assignment</code> , <code>replacement_gets_fuel</code> , <code>is_useless_item</code>
<code>handlers-new/item.lua</code>	<code>item.reflect</code> : turns the matching into the game
<code>compat/vanilla.lua</code>	<code>always_slot_pre</code> , <code>always_slot_dep</code> , first pass blacklist
<code>skeleton/test-monotone-matching.lua</code>	plain-Lua tests for resource slots

B Glossary

Slot / position

The half of a split node that keeps where an item comes from and which recipes use it.

Trav / identity

The half that keeps what the item is and does (places a building, fuel, launch). It’s named **X-trav**.

Matching

A one-to-one assignment of travs to slots of the same type. The vanilla one is the identity matching M_0 .

Pebble, rank

A node reached in a context (room and abilities), and its position in a sort.

Hard pebble

A pebble first pass must keep: a protected mechanic pebble, a planet-locked recipe pebble (both exact), or a recipe (anywhere).

Need

A pair (t, c) : identity t is used in context c by a proof of the hard pebbles.

Rank rule

A needed identity may only move to a slot reached earlier than itself in each needed context.

Gate

A fresh sort of the proposed graph, checking that no hard pebble was lost.

Refinement

After a failed gate, the blocked identity pebbles from the lost pebbles' old witnesses are added as needs, and the round proposes again.

Reservation (greedy only)

A slot/trav pair that's been chosen but not connected, so it adds nothing to the world until it helps a targeted mechanic.